

JARVIS AI ASSISTANT — COMPLETE PROJECT NOTES

Project Architecture, Features, Code Workflow & Bug Fix Explanations

1. System Overview (Kya Hai Yeh Project?)

Jarvis AI Assistant ek advanced voice and text controlled AI system hai jo user ke PC par commands execute karta hai. Isme **Dual Interface** milta hai: Ek desktop CustomTkinter Cyberpunk window aur ek modern HTML5 Holographic Web HUD server. Yeh Groq Llama 3.3 70B Model, Whisper Speech Recognition, aur Edge-TTS engine ka upayog karke real-time Hinglish conversation karta hai.

Component	Technology Used	Function / Use Case
LLM Brain	Groq (Llama-3.3-70b)	Natural language understanding, Hinglish replies, JSON action generation.
Speech-To-Text (STT)	Groq Whisper v3 / Google STT	Voice audio ko text me convert karta hai dynamic energy VAD ke sath.
Text-To-Speech (TTS)	Edge-TTS / pyttsx3	Natural human-like voice synthesis (GuyNeural voice) offline fallback k sath.
Desktop GUI	CustomTkinter & Tkinter	Cyberpunk HUD dashboard, real-time waveform visualizer, system metrics.
Web HUD Server	Python http.server + Web Speech	Browser command center (http://localhost:8000) with 3D canvas orb animation.
System Control	PyAutoGUI, OS, Subprocess, Pycaw	Volume control, app opening, screenshots, system lock/shutdown.

2. Codebase Structure (Files and Modules)

Project ki directory structure aur har main file ka role niche diya gaya hai:

- **main.py** — Main entry point. Desktop GUI aur Web Server ko start karta hai, continuous mic listening loop hold karta hai.
- **web_server.py** — Lightweight HTTP server jo /api/status, /api/metrics, aur /api/command endpoints provide karta hai.
- **config.py** — Environment variables (.env file loading), API Keys, Model configurations, Contacts dictionary aur Default Prompts handle karta hai.
- **core/llm_brain.py** — Groq Llama 3.3 integration, conversation memory (history), aur Hinglish System Prompt ke sath JSON <ACTION> parser.
- **core/stt_engine.py** — Microphone calibration, background noise cancellation, Whisper Large v3 aur Google STT speech recognition.
- **core/tts_engine.py** — Thread-safe Microsoft Edge-TTS voice generation + Pygame audio playback + pyttsx3 offline fallback.
- **core/wake_word.py** — Hands-free wake word detector ('Hey Jarvis', 'Aura') jo continuous background monitoring karta hai.
- **automation/system_control.py** — Windows automation routines (App launcher, volume control via Pycaw, screenshot capture, PC lock/shutdown).
- **gui/app_window.py** — CustomTkinter dark cyberpunk GUI containing sidebar metrics, audio spectrum canvas, and quick action buttons.
- **web/index.html** — Interactive web interface with 60 FPS HTML5 canvas quantum orb visualizer.

3. Executive Workflow (Kaise Kaam Karta Hai?)

- **Step 1: Input Trigger** — User voice microphone se bolta hai ya GUI / Web HUD me text entry dwara command bhejta hai.
- **Step 2: Speech Recognition** — SpeechToTextEngine audio input record karke Groq Whisper Large v3 model ko bhejta hai jo high accuracy text me convert karta hai.
- **Step 3: Intent & Action Analysis** — Text llm_brain.py ko pass hota hai. LLM user query ke aadhar par response frame karta hai. Agar system action required hai (jaise 'open youtube' ya 'mute volume'), toh LLM JSON tag append karta hai:
Mummy ko message bhej raha hu. <ACTION>{"type": "whatsapp", "contact": "mummy", "message": "aa raha hu"}</ACTION>
- **Step 4: Execution & Feedback** — System automation module JSON payload read karke task run karta hai aur tts_engine.py natural voice se bolkar user ko confirmation deta hai.

4. Key Errors Identified and Fixed (Bug Fixes Summary)

Bug / Error Issue	Root Cause	Applied Fix Solution
Wake Word Context Error <i>AssertionError: Audio source inside context manager</i>	Microphone context outer loop me lock hone ki wajah se double-entry exception aa rahi thi.	wake_word.py me microphone context lifecycle ko retry loop ke andar safely isolate kiya.
Corrupt MP3 Playback <i>music_drmp3: corrupt mp3 file (bad tags)</i>	Edge-TTS connection lag ki wajah se incomplete/0-byte MP3 save kar raha tha.	tts_engine.py me os.path.getsize(tmp_path) > 100 check add karke automatic offline fallback lagaya.
Missing Contact Dictionary <i>Contact dictionary mein nahi mila</i>	config.py me CONTACTS mapping dict missing tha.	config.py me default CONTACTS map insert karke contact lookup error solve kiya.
Web HUD Action Handling <i>Commands not executing from Web UI</i>	web_server.py LLM se aaye JSON <ACTION> tag ko extract karke execute nahi kar raha tha.	web_server.py ke /api/command handler me action parser & executor introduce kiya.

5. User Setup & Execution Guide

```
# 1. Environment file (.env) setup:
GROQ_API_KEY=gsk_your_groq_api_key_here

# 2. Install Dependencies:
pip install -r requirements.txt

# 3. Run Jarvis Desktop Assistant & Web HUD:
python main.py

# 4. Access Web Interface:
Open browser at http://localhost:8000
```